Aim: To implement Regression in Python.

Theory:

Q1. What is Regression?

Regression is a statistical and machine learning technique used to model the relationship between a dependent variable (output) and one or more independent variables (inputs), mainly for prediction and trend analysis.

Q2. List applications of Regression.

- • Predicting house prices
- • Sales and revenue forecasting
- • Stock market trend analysis
- • Demand prediction in businesses
- • Risk assessment in finance
- • Weather forecasting
- • Healthcare outcome prediction

Q3. List and explain any 3 types of Regression.

- **1. Linear Regression:**
  Models a straight-line relationship between input and output.
  Example: Predicting salary based on experience.
- **2. Logistic Regression:**
  Used for classification problems (outputs probability between 0 and 1).
  Example: Spam vs Not Spam.
- **3. Ridge Regression:**
  An extension of linear regression that adds a penalty to reduce overfitting.
  Example: Used when data has noise or multicollinearity.

Q4. List metrics to evaluate a regression model.

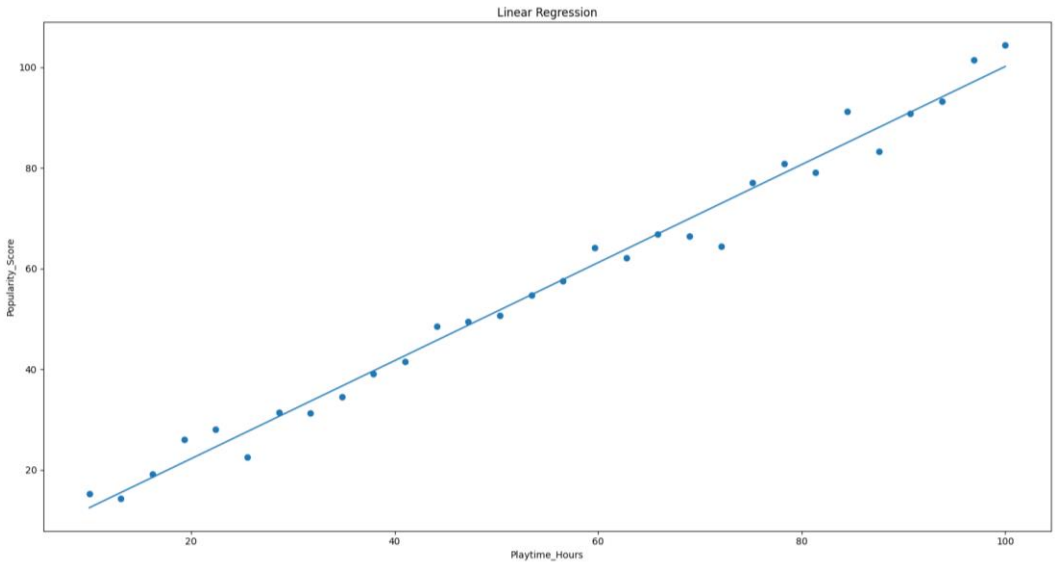
Metric	Meaning
MAE (Mean Absolute Error)	Average absolute difference between actual and predicted values
MSE (Mean Squared Error)	Average of squared errors
RMSE (Root Mean Squared Error)	Square root of MSE (same units as output)
R^2 Score	Proportion of variance explained by the model
Adjusted R^2	Modified R^2 considering number of features

Q5. Write a Python program to implement the Linear Regression.

Code:

```
df = pd.read_csv('games_data.csv')

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
X = df[[num_cols[0]]].values
y = df[num_cols[1]].values

X_b = np.c_[np.ones((len(X),1)), X]
theta = np.linalg.pinv(X_b.T @ X_b) @ X_b.T @ y
y_pred = X_b @ theta

plt.figure()
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.title('Linear Regression')
plt.xlabel(num_cols[0])
plt.ylabel(num_cols[1])
plt.show()

R = np.corrcoef(X.flatten(), y)[0,1]
print("R =", R)
```

Output:




```
spectre@ghouled-gadget MINGW64 /d/degree/SEM 6/DMBI/Experiments/EXP8 (master)
$ py linear.py
R = 0.9927386796016454
```

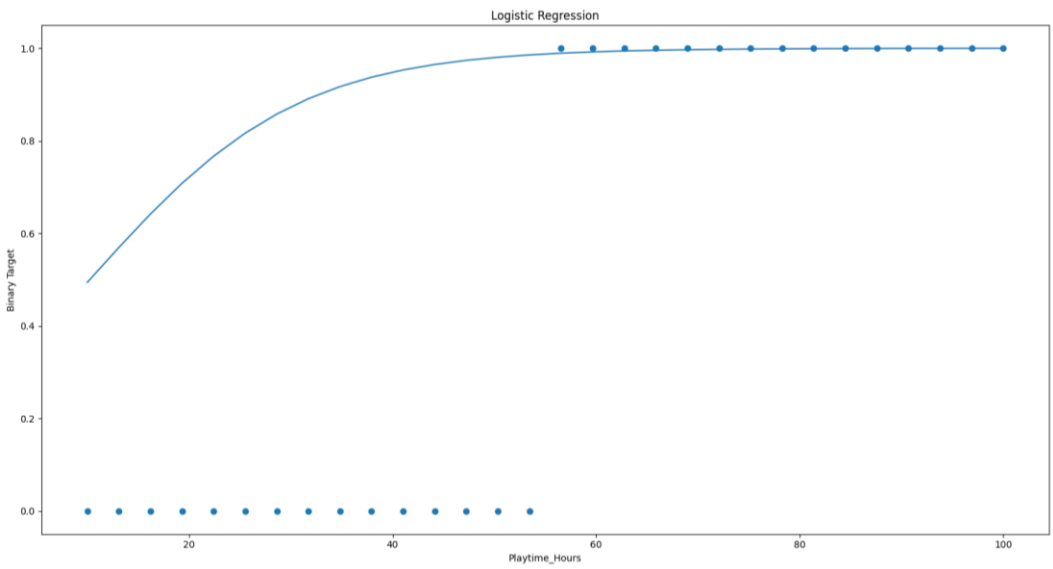
Q6. Write a Python program to implement the Logistic Regression.

Code:

```
df = pd.read_csv('games_data.csv')

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
X = df[[num_cols[0]]].values
```

```
y = df[num_cols[1]].values

y_bin = (y > np.median(y)).astype(int)

X_b = np.c_[np.ones((len(X),1)), X]
w = np.zeros(X_b.shape[1])
lr = 0.01

for _ in range(500):
    z = X_b @ w
    h = 1/(1+np.exp(-z))
    grad = X_b.T @ (h - y_bin) / len(y_bin)
    w -= lr * grad

y_pred = 1/(1+np.exp(-(X_b @ w)))

plt.figure()
plt.scatter(X, y_bin)
plt.plot(X, y_pred)
plt.title('Logistic Regression')
plt.xlabel(num_cols[0])
plt.ylabel('Binary Target')
plt.show()

R = np.corrcoef(X.flatten(), y)[0,1]
print("R =", R)
```

Output:



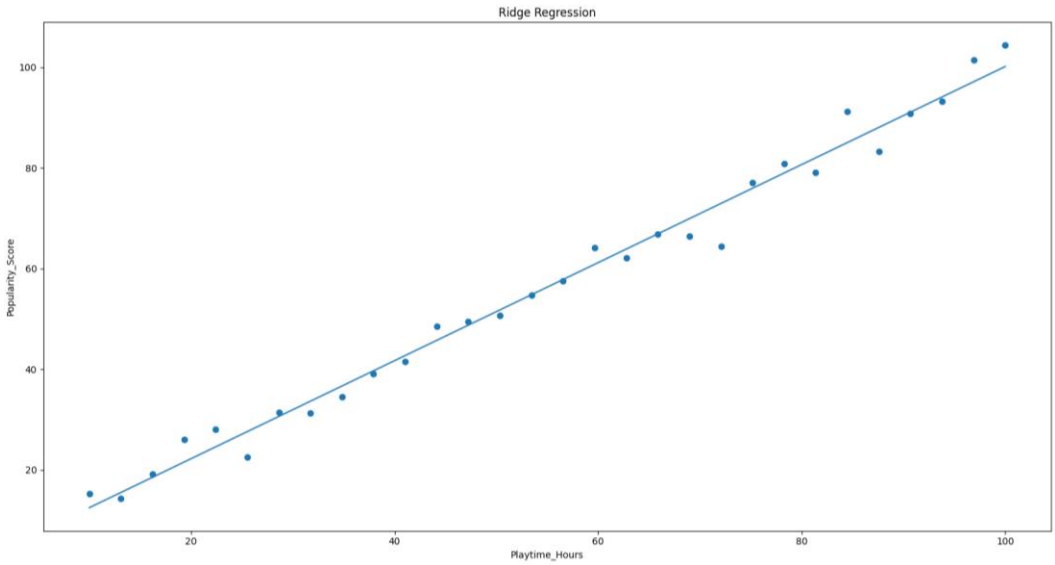

```
spectre@ghouled-gadget MINGW64 /d/degree/SEM 6/DMBI/Experiments/EXP8 (master)
$ py logistic.py
R = 0.9927386796016454
```

Q7. Write a Python program to implement Ridge Regression.

Code:

```
df = pd.read_csv('games_data.csv')

num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
X = df[[num_cols[0]]].values
y = df[num_cols[1]].values

X_b = np.c_[np.ones((len(X),1)), X]
lam = 1.0
I = np.eye(X_b.shape[1]); I[0,0] = 0
theta = np.linalg.pinv(X_b.T @ X_b + lam*I) @ X_b.T @ y
y_pred = X_b @ theta

plt.figure()
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.title('Ridge Regression')
plt.xlabel(num_cols[0])
plt.ylabel(num_cols[1])
plt.show()

R = np.corrcoef(X.flatten(), y)[0,1]
print("R =", R)
```

Output:




```
spectre@ghouled-gadget MINGW64 /d/degree/SEM 6/DMBI/Experiments/EXP8 (master)
$ py ridge.py
R = 0.9927386796016454
```

Conclusion:

The Apriori algorithm effectively uncovers hidden purchasing patterns, enabling data-driven cross-selling strategies and optimized product placements.